EF Core 8.0

Lab 1: Understanding ORM with a Retail Inventory System

Scenario:

You're building an inventory management system for a retail store. The store wants to track products, categories, and stock levels in a SQL Server database.

Objective:

Understand what ORM is and how EF Core helps bridge the gap between C# objects and relational tables.

Model Classes

Models/Category.cs

```
using System.Collections.Generic;

namespace RetailInventory.Models
{
    public class Category
    {
        public int CategoryId { get; set; }
        public string Name { get; set; }
        public string Description { get; set; }
        public List<Product> Products { get; set; } = new List<Product>();
    }
}
```

Models/Product.cs

```
namespace RetailInventory.Models
{
    public class Product
    {
        public int ProductId { get; set; }
        public string Name { get; set; }
```

```
        public decimal Price { get; set; }
        public int StockQuantity { get; set; }
        public int CategoryId { get; set; }
        public Category Category { get; set; }
    }
}
```

Data/RetailContext.cs

```
using Microsoft.EntityFrameworkCore;
using RetailInventory.Models;
namespace RetailInventory.Data
{
    public class RetailContext : DbContext
    {
        public DbSet<Product> Products { get; set; }
        public DbSet<Category> Categories { get; set; }
        protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
        {
            optionsBuilder.UseSqlServer(

@"Server=(localdb)\MSSQLLocalDB;Database=RetailInventoryDb;Trusted_C
onnection=True;TrustServerCertificate=True;");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<Product>()
                .Property(p => p.Price)
                .HasColumnType("decimal(18,2)");
```

```
        modelBuilder.Entity<Product>()
            .HasOne(p => p.Category)
            .WithMany(c => c.Products)
            .HasForeignKey(p => p.CategoryId);  }}}
```

Program.cs

```
using System;
using System.Linq;
using Microsoft.EntityFrameworkCore;
using RetailInventory.Data;
using RetailInventory.Models;

namespace RetailInventory
{
   class Program
   {
      static void Main(string[] args)
      {
         using (var context = new RetailContext())
         {
            context.Database.EnsureCreated();
            SeedData(context);
            DisplayAllProducts(context);
            QueryLowStockProducts(context);
            UpdateStock(context);
         }
```

```
        Console.WriteLine("\nDone. Press any key to exit...");
        Console.ReadKey();
    }
    static void SeedData(RetailContext context) {
        if (!context.Categories.Any())
        {
            var electronics = new Category { Name = "Electronics", Description =
"Gadgets and devices" };
            var groceries = new Category { Name = "Groceries", Description =
"Everyday food items" };


            context.Categories.AddRange(electronics, groceries);
            context.SaveChanges();
            context.Products.AddRange(
                new Product { Name = "Wireless Mouse", Price = 799.00m,
StockQuantity = 50, CategoryId = electronics.CategoryId },
                new Product { Name = "Bluetooth Speaker", Price = 1999.00m,
StockQuantity = 15, CategoryId = electronics.CategoryId },
                new Product { Name = "Rice (5kg)", Price = 450.00m,
StockQuantity = 100, CategoryId = groceries.CategoryId },
                new Product { Name = "Cooking Oil (1L)", Price = 180.00m,
StockQuantity = 8, CategoryId = groceries.CategoryId }
            );
            context.SaveChanges();
            Console.WriteLine("Seed data inserted.\n");
        }
    }
    static void DisplayAllProducts(RetailContext context)
    {
```

```
        Console.WriteLine("=== All Products ===");
        var products = context.Products
                        .Include(p => p.Category)
                        .ToList();
        foreach (var p in products)
        {
            Console.WriteLine($"{p.ProductId}. {p.Name} | Category:
{p.Category.Name} | Price: {p.Price:C} | Stock: {p.StockQuantity}");
        }
    }

    static void QueryLowStockProducts(RetailContext context)
    {
        Console.WriteLine("\n=== Low Stock Products (Stock < 20) ===");
        var lowStock = context.Products
                        .Where(p => p.StockQuantity < 20)
                        .ToList();
        foreach (var p in lowStock)  {
            Console.WriteLine($"{p.Name} - only {p.StockQuantity} left!");
        }
    }
    static void UpdateStock(RetailContext context)    {
        var mouse = context.Products.FirstOrDefault(p => p.Name == "Wireless
Mouse");
        if (mouse != null)
        {
            mouse.StockQuantity -= 5;
            context.SaveChanges();
```

Console.WriteLine($"\nUpdated stock for {mouse.Name}: {mouse.StockQuantity} remaining."); }}}}

Output

```
Seed data inserted.

=== All Products ===
1. Wireless Mouse | Category: Electronics | Price: ₹799.00 | Stock: 50
2. Bluetooth Speaker | Category: Electronics | Price: ₹1,999.00 | Stock: 15
3. Rice (5kg) | Category: Groceries | Price: ₹450.00 | Stock: 100
4. Cooking Oil (1L) | Category: Groceries | Price: ₹180.00 | Stock: 8

=== Low Stock Products (Stock < 20) ===
Bluetooth Speaker - only 15 left!
Cooking Oil (1L) - only 8 left!

Updated stock for Wireless Mouse: 45 remaining.

Done. Press any key to exit...
```